Program Calculation with Hylomorphisms in Rocq

David Castro-Perez

David.CastroPerez@rhul.ac.uk



Based on joint work with Marco Paviotti and Michael Vollmer (University of Kent):
Program Optimisations via Hylomorphisms for Extraction of Executable Code, ITP 2025.

Program calculation:

- Simple specification
- Rewrite with laws
- Efficient program

Our work:

- Hylomorphisms in Rocq
- Axiom-free
- Idiomatic OCaml extraction

D. Castro-Perez, M. Paviotti, M. Vollmer. **Program Optimisations via Hylomorphisms for Extraction of Executable Code**. ITP 2025. Library: `https://github.com/dcastrop/rocq-hylomorphisms`

A Specification and a Program

$$\mathsf{map}\,(\lambda x.\, 2x) \circ \mathsf{sort}$$

```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

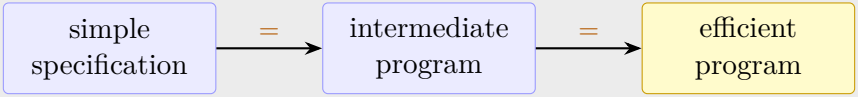
A Specification and a Program

$$\mathsf{map}\,(\lambda x.\, 2x) \circ \mathsf{sort}$$

```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

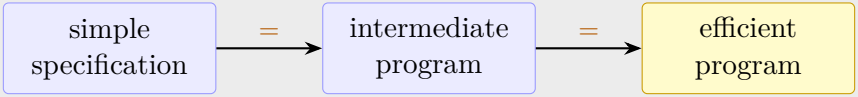
Can we *derive* the program from the specification?

Program Calculation



- Every step is an equation: correct by construction.

Program Calculation



- Every step is an equation: correct by construction.
- The laws come from ***recursion schemes***.

Background

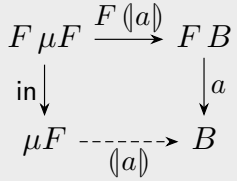
Recursion Schemes

Functors and Algebras

$$\mathsf{List}\, A \cong 1 + A \times \mathsf{List}\, A = F\,(\mathsf{List}\, A)$$

- *Algebra*: $a : F\, B \to B$.
- *Initial algebra*: the constructors, $\mathsf{in} = [\mathsf{nil}, \mathsf{cons}]$.

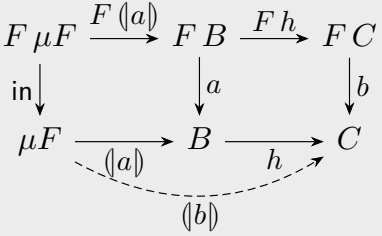
Catamorphisms (Folds)



$$f = (\!|a|\!) \iff f \circ \mathsf{in} = a \circ F\,f$$

A *unique* arrow out of the initial algebra.

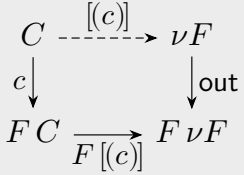
Fusion



$$h \circ a = b \circ F\,h \implies h \circ (\!|a|\!) = (\!|b|\!)$$

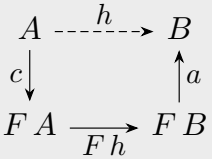
Two traversals become one.

Coalgebras and Anamorphisms (Unfolds)



$$\mathsf{digits}_2\, n = \textbf{if } n = 0 \textbf{ then } \mathsf{nil} \textbf{ else } (n \bmod 2,\ n/2)$$

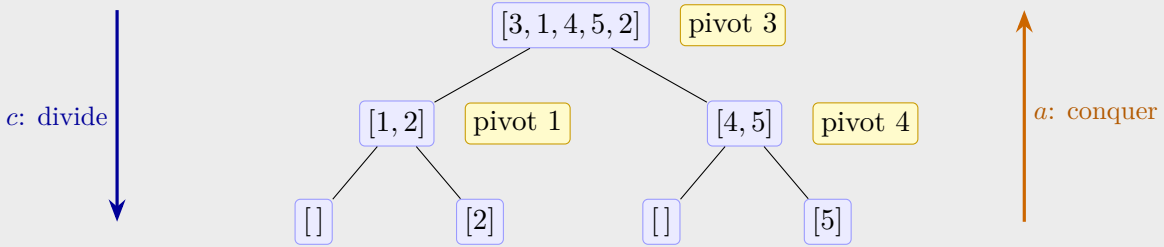
Hylomorphisms: Divide and Conquer



$$h = a \circ F\,h \circ c$$

Quicksort: c partitions, a concatenates.

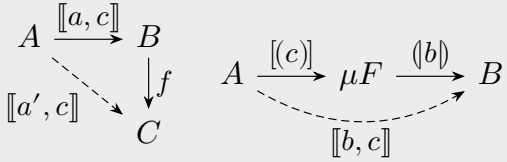
The Call Tree of Quicksort



A hylomorphism never builds this tree.

Recursive Coalgebras and their Laws

c *recursive*: a unique solution $[\![a, c]\!]$ for every a.



$$
\begin{aligned}
f \circ a = a' \circ F\, f \quad &\Longrightarrow \quad f \circ [\![a, c]\!] = [\![a', c]\!] \\
c \circ f = F\, f \circ c' \quad &\Longrightarrow \quad [\![a, c]\!] \circ f = [\![a, c']\!] \\
c' \circ a = \mathsf{id} \quad &\Longrightarrow \quad [\![b, c']\!] \circ [\![a, c]\!] = [\![b, c]\!]
\end{aligned}
$$

Conjugate Hylomorphisms

Every recursion scheme is a conjugate hylomorphism

recursion scheme	*adjunction*	*conjugates*	*para-hylo equation*	*algebra*
(hylo-shift law)	$\mathsf{Id} \dashv \mathsf{Id}$	$\alpha \dashv \alpha$	$x = a \cdot (id \vartriangle \mathsf{D}\, x \cdot \alpha\, C \cdot c)\ :\ A \leftarrow C$	$a : C \times \mathsf{D}\, A \to A$
mutual recursion	$\Delta \dashv (\times)$	ccf	$x_1 = a_1 \cdot (id \vartriangle \mathsf{D}\, (x_1 \vartriangle x_2) \cdot c)\ :\ A_1 \leftarrow C$ $x_2 = a_2 \cdot (id \vartriangle \mathsf{D}\, (x_1 \vartriangle x_2) \cdot c)\ :\ A_2 \leftarrow C$	$a_1 : C \times \mathsf{D}\, (A_1 \times A_2) \to A_1$ $a_2 : C \times \mathsf{D}\, (A_1 \times A_2) \to A_2$
accumulator	$- \times P \dashv (-)^P$	ccf	$x = a \cdot (outl \vartriangle ((\mathsf{D}\, (\wedge\, x) \cdot c) \times P))\ :\ A \leftarrow C \times P$	$a : C \times \mathsf{D}\, (A^P) \times P \to A$
course-of-values (§5.6)	$\mathsf{U}_{\mathsf{D}} \dashv \mathsf{Cofree}_{\mathsf{D}}$	ccf	$x = a \cdot (id \vartriangle \mathsf{D}\, (\mathsf{D}_\infty\, x \cdot \llparenthesis c \rrparenthesis) \cdot c)\ :\ A \leftarrow C$	$a : C \times \mathsf{D}\, (\mathsf{D}_\infty\, A) \to A$
finite memo-table (§5.6)	$\mathsf{U}_* \dashv \mathsf{Cofree}_*$	ccf	$x = a \cdot (id \vartriangle \mathsf{D}\, (\mathsf{D}_*\, x \cdot \llparenthesis c \rrparenthesis_*) \cdot c)\ :\ A \leftarrow C$	$a : C \times \mathsf{D}\, (\mathsf{D}_*\, A) \to A$

Table 1. Different types of para-hylos building on the canonical control functor (ccf); the coalgebra is $c : C \to \mathsf{D}\, C$ in each case.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

Why Hylomorphisms in Rocq?

- Haskell: the schemes, but no termination.
- Rocq: termination, but these definitions are rejected.
- We want: a ***library***, ***extracted code***, and the ***laws as tactics***.

Challenges

1. **No axioms**
2. **Clean extracted code**
3. **Non-structural recursion**

Challenges

1. **No axioms**
2. **Clean extracted code**
3. **Non-structural recursion**

Solutions:

1. Setoids; decidable validity.

Challenges

1. **No axioms**
2. **Clean extracted code**
3. **Non-structural recursion**

Solutions:

1. Setoids; decidable validity.
2. One type of positions, not a family.

Challenges

1. **No axioms**
2. **Clean extracted code**
3. **Non-structural recursion**

Solutions:

1. Setoids; decidable validity.
2. One type of positions, not a family.
3. Recursion on a termination proof.

Challenges

1. **No axioms**
2. **Clean extracted code**
3. **Non-structural recursion**

Solutions:

1. Setoids; decidable validity.
2. One type of positions, not a family.
3. Recursion on a termination proof.

Since the paper: usability.

Roadmap

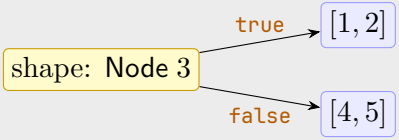
Part I:	Extractable Containers
Part II:	Recursive Coalgebras
Part III:	Calculation and Extraction
Part IV:	Dynamic Programming
Part V:	Monoid Algebras

Part I

Extractable Containers in Rocq

Containers

$$[\![S \lhd P]\!]\, X = \sum_{s:S} (P\, s \to X) \qquad [\![S \lhd P]\!]\, f\, (s, g) = (s, f \circ g)$$



shape: Leaf no positions

M. Abbott, T. Altenkirch, N. Ghani. **Categories of Containers**. FoSSaCS 2003.

Containers in Rocq: A Bad Attempt

```
Record Container := { Sh : Type; Pos : Sh -> Type }.
Record Ext C X := MkExt { shape : Sh C; cont : Pos C shape -> X }.

(* quicksort: a leaf, or a pivot with two positions *)
Definition TreeC : Container :=
  {| Sh := option nat;
     Pos := fun s => match s with None => False | Some _ => bool end |}.
```

Containers in Rocq: A Bad Attempt

```
Record Container := { Sh : Type; Pos : Sh -> Type }.
Record Ext C X := MkExt { shape : Sh C; cont : Pos C shape -> X }.

(* quicksort: a leaf, or a pivot with two positions *)
Definition TreeC : Container :=
  {| Sh := option nat;
     Pos := fun s => match s with None => False | Some _ => bool end |}.
```

Positions are a type *family*.

A Bad Attempt: Extraction and Equality

```
type sh = __                                    (* = Obj.t *)
type pos = __
type 'x ext = { shape : sh; cont : (pos -> 'x) }

let merge x =
  let { shape = shape0; cont = k } = x in
  (match Obj.magic shape0 with
   | Some h -> app (Obj.magic k True) (Cons (h, (Obj.magic k False)))
   | None -> Nil)
```

A Bad Attempt: Extraction and Equality

```
Lemma ext_inj s (k1 k2 : Pos C s -> X) : MkExt s k1 = MkExt s k2 -> k1 = k2.
Proof. intros H; injection H; intros E.
  (* E : existT _ s k1 = existT _ s k2.  Now what? *)
  apply inj_pair2 in E; exact E. Qed.
```

```
Print Assumptions ext_inj.  Axioms: Eq_rect_eq.eq_rect_eq   (axiom K)
Print Assumptions fmap_id.  Axioms: functional_extensionality_dep
```

Extractable Containers in Rocq (I)

1. Decidable equality gives UIP: no axiom.
2. Proofs in `Prop` are erased.

Extractable Containers in Rocq (I)

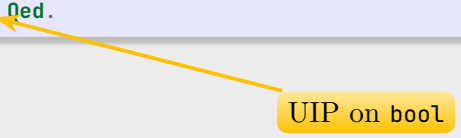
1. Decidable equality gives UIP: no axiom.
2. Proofs in `Prop` are erased.

```
Lemma elem_val_eq (e1 e2 : Pos F s) : val e1 = val e2 -> e1 = e2.
Proof. ...; apply bool_irrelevance. Qed.
```

Extractable Containers in Rocq (I)

1. Decidable equality gives UIP: no axiom.
2. Proofs in `Prop` are erased.

```
Lemma elem_val_eq (e1 e2 : Pos F s) : val e1 = val e2 -> e1 = e2.
Proof. ...; apply bool_irrelevance. Qed.
```



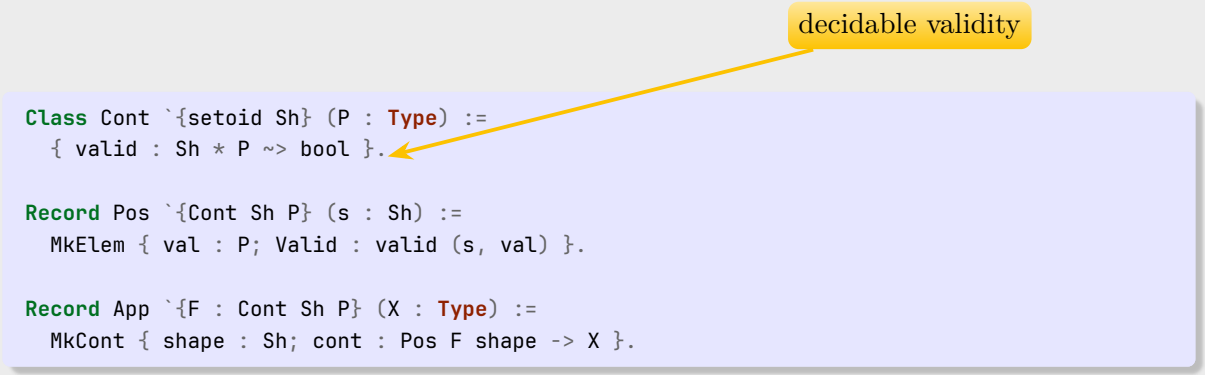
Extractable Containers in Rocq (and II)

```
Class Cont `{setoid Sh} (P : Type) :=
  { valid : Sh * P ~> bool }.

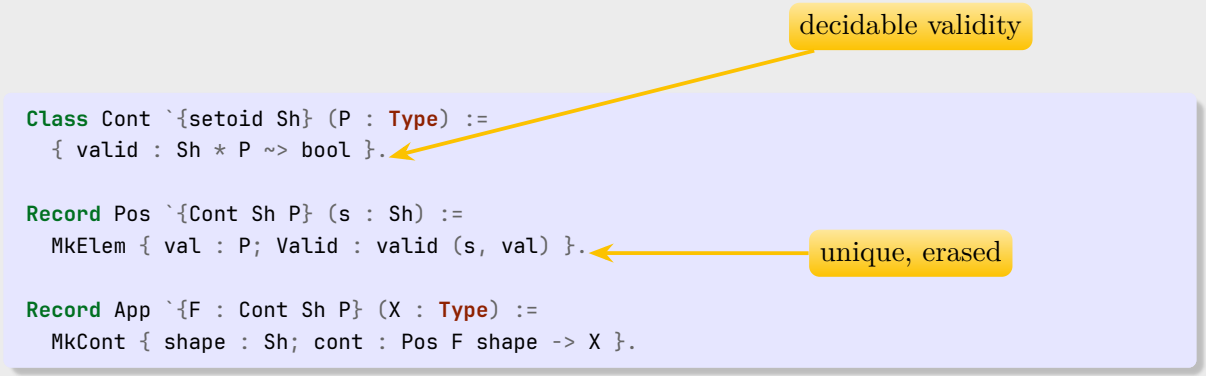
Record Pos `{Cont Sh P} (s : Sh) :=
  MkElem { val : P; Valid : valid (s, val) }.

Record App `{F : Cont Sh P} (X : Type) :=
  MkCont { shape : Sh; cont : Pos F shape -> X }.
```

Extractable Containers in Rocq (and II)



Extractable Containers in Rocq (and II)




```
Class Cont `{setoid Sh} (P : Type) :=
  { valid : Sh * P ~> bool }.

Record Pos `{Cont Sh P} (s : Sh) :=
  MkElem { val : P; Valid : valid (s, val) }.

Record App `{F : Cont Sh P} (X : Type) :=
  MkCont { shape : Sh; cont : Pos F shape -> X }.
```

Example: the Recursion of Quicksort

```
Inductive Ts L A := Leaf (l : L) | Node (a : A).        (* shapes    *)
Inductive Tp := Lbranch | Rbranch.                      (* positions *)

Definition t_dom (x : Ts L A * Tp) : bool :=
  match x with (Node _, _) => true | _ => false end.
Instance TreeF L A : Cont (Ts L A) Tp := { valid := t_dom }.
```

t_dom	Lbranch	Rbranch
Leaf tt	false	false
Node h	true	true

Setoids and Morphisms

- Every type has an equivalence `=e`.
- `A ~> B`: functions that respect it.
- Laws become `rewrite` steps.

Part I in One Slide

Problem	**Cause**	**Fix**
`Obj.magic`	dependent positions	one type of positions
axiom K	dependent pairs	boolean validity
function extensionality	`=` on functions	setoids

Part II

Recursive Coalgebras and Hylomorphisms

Why Not Just Write Quicksort?

```
Fixpoint qsort (l : list nat) : list nat :=
  match l with
  | [] => []
  | h :: t => qsort (filter (fun x => x <? h) t)
              ++ h :: qsort (filter (fun x => negb (x <? h)) t)
  end.
```

```
Error: Recursive call to qsort has principal argument equal to
"filter (fun x : nat => negb (x <? h)) t" instead of "t".
```

Recursive Coalgebras (I)

$$h = a \circ F\, h \circ c \qquad \text{with } F\, X = X, \text{ so } F\, h = h$$

J. Adámek, S. Milius, L. S. Moss. **On Well-Founded and Recursive Coalgebras**. FoSSaCS 2020.
V. Capretta, T. Uustalu, V. Vene. **Recursive Coalgebras from Comonads**. 2006.

Recursive Coalgebras (I)

$$h = a \circ F\, h \circ c \qquad \text{with } F\, X = X, \text{ so } F\, h = h$$

$$c = \mathsf{id},\ a = \mathsf{succ} \qquad h\, x = h\, x + 1 \qquad \textbf{no solution}$$

J. Adámek, S. Milius, L. S. Moss. **On Well-Founded and Recursive Coalgebras**. FoSSaCS 2020.
V. Capretta, T. Uustalu, V. Vene. **Recursive Coalgebras from Comonads**. 2006.

Recursive Coalgebras (I)

$$h = a \circ F\, h \circ c \qquad \text{with } F\, X = X, \text{ so } F\, h = h$$

$c = \mathsf{id},\ a = \mathsf{succ}$	$h\, x = h\, x + 1$	**no** solution
$c = a = \mathsf{id}$	$h\, x = h\, x$	**every** h is a solution

J. Adámek, S. Milius, L. S. Moss. **On Well-Founded and Recursive Coalgebras**. FoSSaCS 2020.
V. Capretta, T. Uustalu, V. Vene. **Recursive Coalgebras from Comonads**. 2006.

Recursive Coalgebras (I)

$$h = a \circ F\, h \circ c \qquad \text{with } F\, X = X, \text{ so } F\, h = h$$

$c = \mathsf{id},\ a = \mathsf{succ}$	$h\, x = h\, x + 1$	**no** solution
$c = a = \mathsf{id}$	$h\, x = h\, x$	**every** h is a solution

The culprit: $c = \mathsf{id}$ never shrinks its input, $x \mapsto x \mapsto x \mapsto \cdots$

Recursive c: every input reaches the leaves; then there is exactly one h, for every a.

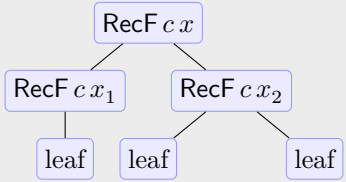
J. Adámek, S. Milius, L. S. Moss. **On Well-Founded and Recursive Coalgebras**. FoSSaCS 2020.
V. Capretta, T. Uustalu, V. Vene. **Recursive Coalgebras from Comonads**. 2006.

Recursive Coalgebras (and II)

```
Inductive RecF (c : Coalg F A) : A -> Prop :=
| RecF_fold x : (forall e, RecF c (cont (c x) e)) -> RecF c x.

Definition RecP (c : Coalg F A) := forall x, RecF c x.

Structure RCoalg F A := Rec { coalg :> Coalg F A; recP : RecP coalg }.
```



Equivalent to `well_founded (child c)`, but the induction hypothesis is *per position*, as in `fmap`.

Recursive Hylomorphisms

Recursion on the termination proof (simplified):

```
Definition hylo_def (a : Alg F B) (c : Coalg F A)
  : forall (x : A), RecF c x -> B :=
  fix f x H :=
    match c x with
    | MkCont sx cx => a (MkCont sx (fun e => f (cx e) (RecF_inv H e)))
    end.
```

Recursive Hylomorphisms

Recursion on the termination proof (simplified):

```
Definition hylo_def (a : Alg F B) (c : Coalg F A)
  : forall (x : A), RecF c x -> B :=
  fix f x H :=
    match c x with
    | MkCont sx cx => a (MkCont sx (fun e => f (cx e) (RecF_inv H e)))
    end.
```

termination proof
(in `Prop`: erased)

Recursive Hylomorphisms

Recursion on the termination proof (simplified):

```
Definition hylo_def (a : Alg F B) (c : Coalg F A)
  : forall (x : A), RecF c x -> B :=
  fix f x H :=
    match c x with
    | MkCont sx cx => a (MkCont sx (fun e => f (cx e) (RecF_inv H e)))
    end.
```

termination proof (in `Prop`: erased)

structurally smaller proof

Uniqueness and the Laws

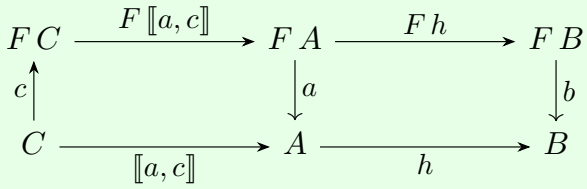
```
Lemma hylo_uniq : f =e hylo a c <-> f =e a \o fmap f \o c.

Lemma hylo_fusion_l : h \o a =e b \o fmap h -> h \o hylo a c =e hylo b c.
Lemma hylo_fusion_r : c \o h =e fmap h \o d -> hylo a c \o h =e hylo a d.
Lemma deforest      : d \o b =e id -> hylo a d \o hylo b c =e hylo a c.
```

Uniqueness and the Laws

```
Lemma hylo_uniq : f =e hylo a c <-> f =e a \o fmap f \o c.
```

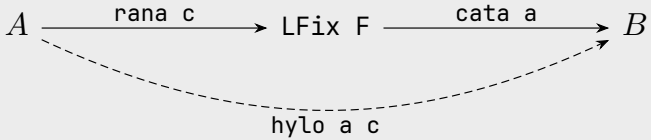
Proofs follow the paper proofs:



Folds after Unfolds

```
Definition rana (c : RCoalg F A) : A ~> LFix F := hylo l_in c.

Corollary cata_ana_hylo : cata a \o rana c =e hylo a c.
```



No Axioms

```
Print Assumptions hylo_fusion_l.          Closed under the global context
Print Assumptions mss_calc.               Closed under the global context
Print Assumptions qsort_times_two_calc.   Axioms: PrimInt63.int, ltb, mul
```

Part III

Program Calculation and Extraction

Calculating a Program with `Derive`

```
Derive qsort in (cata merge \o rana tsplit =e morph_of qsort) as qsort_calc.
Proof.
  rewrite cata_ana_hylo, <- (hylo_cps_hylo merge tsplit_k).
  finished.
Qed.
```

```
let rec qsort = function
| [] -> []
| i :: l ->
  let (l0, l1) = partition (fun x0 -> Uint63.lt x0 i) l in
  app (qsort l0) (i :: (qsort l1))
```

Fusing a Map into Quicksort

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e morph_of qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  rewrite <- (hylo_cps_hylo _ tsplit_k).
  finished.
Qed.
```

```
let rec qsort_times_two = function
| [] -> []
| i :: l ->
  let (l0, l1) = partition (fun x0 -> Uint63.lt x0 i) l in
  app (qsort_times_two l0)
    ((Uint63.mul (Uint63.of_int (2)) i) :: (qsort_times_two l1))
```

Quicksort, Step by Step

```
(Lmap times_two \o cata merge) \o rana tsplit
```

Quicksort, Step by Step

```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
```

Quicksort, Step by Step

```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
=e  (* hylo_ana, deforest: unfold then fold, with no tree in between *)
  hylo (merge \o natural (nt_shape id times_two)) tsplit
```

Quicksort, Step by Step

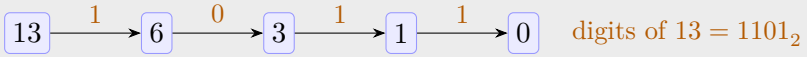
```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
=e  (* hylo_ana, deforest: unfold then fold, with no tree in between *)
  hylo (merge \o natural (nt_shape id times_two)) tsplit
=e  (* hylo_cps_hylo: continuation-passing, for clean extraction *)
  hylo_cps (merge \o natural (nt_shape id times_two)) tsplit_k
```

Coalgebras Need Not Follow the Data

```
Definition digits_ (b n : int) : App (ListF int) int :=
  MkCont (if n =? 0 then s_nil else s_cons (n mod b)) (fun _ => n / b).

Definition power (x : int) : int ~> int :=
  cata (pow_alg x) \o rana (digits 2 eq_refl).

Derive fast_power in (power x =e morph_of fast_power) as fast_power_calc.
```



digits of $13 = 1101_2$

```
let rec fast_power x x0 =
  if Uint63.equal x0 (Uint63.of_int (0))
  then (Uint63.of_int (1))
  else Uint63.mul (digit_pow x (Uint63.rem x0 (Uint63.of_int (2))))
         (sq (fast_power x (Uint63.div x0 (Uint63.of_int (2)))))
```

Challenge: Inlining for Extraction

Extracting the unreduced `hylo merge tsplit`:

```
type ('sh, 'p, 'x) app0 = { shape : 'sh; cont0 : (('sh, 'p) pos -> 'x) }

let rec hylo esh f eA eB g h x =
  let { shape = sx; cont0 = cx } = h x in
  g { shape = sx; cont0 = (fun e -> hylo esh f eA eB g h (cx e)) }

let qsort_spec =
  hylo def_eq treeF (list_eq def_eq) (list_eq def_eq) merge tsplit
```

Challenge: Inlining for Extraction

dictionaries at run time

Extracting the unreduced `hylo merge tsplit`:

```
type ('sh, 'p, 'x) app0 = { shape : 'sh; cont0 : (('sh, 'p) pos -> 'x) }

let rec hylo esh f eA eB g h x =
  let { shape = sx; cont0 = cx } = h x in
  g { shape = sx; cont0 = (fun e -> hylo esh f eA eB g h (cx e)) }

let qsort_spec =
  hylo def_eq treeF (list_eq def_eq) (list_eq def_eq) merge tsplit
```

Challenge: Inlining for Extraction

dictionaries at run time

Extracting the unreduced `hylo merge tsplit`:

```
type ('sh, 'p, 'x) app0 = { shape : 'sh; cont0 : (('sh, 'p) pos -> 'x) }

let rec hylo esh f eA eB g h x =
  let { shape = sx; cont0 = cx } = h x in
  g { shape = sx; cont0 = (fun e -> hylo esh f eA eB g h (cx e)) }

let qsort_spec =
  hylo def_eq treeF (list_eq def_eq) (list_eq def_eq) merge tsplit
```

a record per call

Since the Paper: Usability

- Properness and termination: **instance search**.

Since the Paper: Usability

- Properness and termination: **instance search**.
- Rewriting modulo associativity: `arewrite`.

Since the Paper: Usability

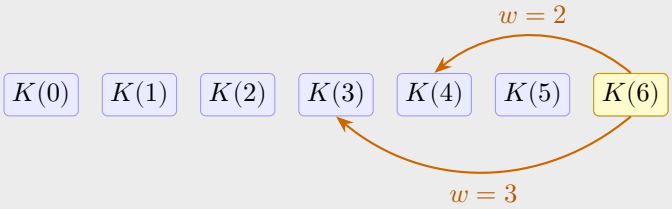
- Properness and termination: **instance search**.
- Rewriting modulo associativity: `arewrite`.
- Diverging instance search: `#[mode = "!"]`.

Part IV

Dynamic Programming

Knapsack

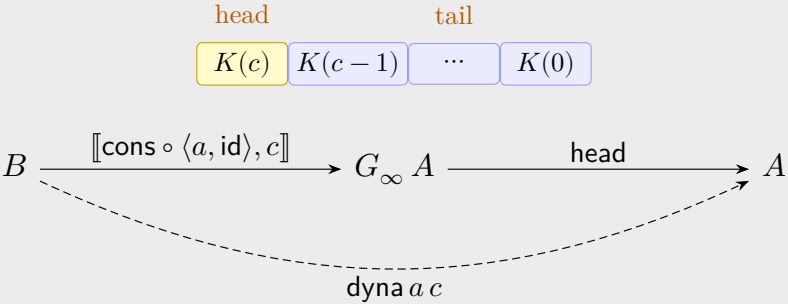
$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \leq c \,\}\right)$$



Items (w, v): $(2, 3)$ and $(3, 4)$, so $K(6) = \max(K(4) + 3, K(3) + 4) = 9$.

The naive hylomorphism recurses over the items: exponential.

Dynamorphisms: Algebras with a Memory



Dynamorphisms in Rocq

```
Instance Memo : Cont (A * Sg) Pg := ...                (* Memo X = A * G X *)

Definition dyna `{!Initial (Memo G A) T}
  (a : App G T ~> A) (c : RCoalg G B) : B ~> A
  := mhead \o l_out \o hylo (l_in \o mcons \o pair a id) c.

Theorem dyna_iso : dyna a c =e dyna (a \o fmap (iso U T)) c.  (* any table *)

Instance ATab_initial : Initial (Memo NatF A) (ATab A).    (* arrays *)
```

Changing the Recursion

```
Theorem reenumerate (an : App F A ~> A) (cn : RCoalg F B)
  (e : RCoalg NatF B) (alg : (int -> option A) ~> A)
  (Hlook : forall (g : B ~> A) b,
      an (fmap g (cn b)) =e alg (look_rec e g b))
  : hylo an cn =e dyna (alg \o look_with) e.
```

- `an`, `cn`: the naive recursion (over items).
- `e`: the recursion we want (count down capacities).
- `Hlook`: `an` only needs results that `e` reaches.
- `look_with`: lookups jump into the array table, $O(1)$.

Knapsack: the Calculation

```
Definition knapsack_naive wvs := hylo naive_alg (naive_c wvs).  (* exponential *)

(* Step 1: the algebra reads its subproblems from the table *)
Derive knap_alg_ in (forall g b, naive_alg (fmap g (naive_c wvs b))
                               = knap_alg_ (look_rec out_int_c g b))
  as knap_look.

(* Step 2: the loop's step, from the table *)
Derive knap_step in (knap_alg \o look_with fast_drop_walk \o nat_next
                     =e knap_step) as knap_step_calc.
Proof. finished. Qed.
```

Step 1's proof is left out.

Knapsack: the Program

```
Derive knapsack_arr in (knapsack_naive wvs =e morph_of knapsack_arr)
  as knapsack_arr_calc.
Proof.
  unfold knapsack_naive.
  (* Memoise: the subproblems are read from a table *)
  erewrite reenumerate.
  (* The table of a count-down is built by a loop *)
  erewrite dyna_loop.
  (* The loop's step, its first entry, and the algebra on the table *)
  2: exact knap_step_calc.
  2: exact knap_base.
  2: exact knap_look.
  finished.
Qed.
```

Knapsack, Step by Step

```
knapsack_naive wvs
```

Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
```

Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
=e  (* reenumerate: recurse on capacities, look results up in O(1) *)
  dyna (knap_alg wvs \o look_with (w:=fast_drop) fast_drop_walk) out_int_c
```

Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
=e  (* reenumerate: recurse on capacities, look results up in O(1) *)
  dyna (knap_alg wvs \o look_with (w:=fast_drop) fast_drop_walk) out_int_c
=e  (* dyna_loop: the table of a count-down is built by a loop *)
  (mhead \o l_out) \o
    (morphism fun n : int =>
       loop n (fun _ : int => memo_step (knap_step wvs))
         (0%uint63, memo_base 0%uint63))
```

Knapsack: What Comes Out

```
let knap_step wvs x =
  imax (Uint63.of_int (0))
    (let rec f = function
     | [] -> min_int
     | h :: t ->
       imax
         (match fast_drop (fst h) x with
          | Some a -> Uint63.add (head a) (snd h)
          | None -> min_int)
         (f t)
     in f wvs)

let knapsack_arr wvs x =
  head
    (let rec f = function
     | (i, s) ->
       if Uint63.lt i x
       then f ((Uint63.add i (Uint63.of_int (1))), (push (knap_step wvs s) s))
       else s
     in f ((Uint63.of_int (0)), (single (Uint63.of_int (0)))))
```

Knapsack: What Comes Out

```
let rec jump n t =
  let { blk = b; top = i; prev = p } = t in
  if Uint63.le n i
  then Some { blk = b; top = (Uint63.sub i n); prev = p }
  else (match p with
        | Some q -> jump (Uint63.sub (Uint63.sub n i) (Uint63.of_int (1))) q
        | None -> None)

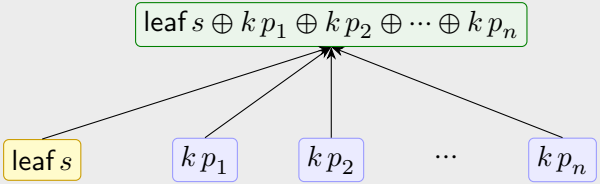
let fast_drop j t =
  if Uint63.equal j (Uint63.of_int (0))
  then None
  else jump (Uint63.sub j (Uint63.of_int (1))) t
```

`fast_drop`: jump straight into the right block of the array, $O(1)$. Knapsack is linear in the capacity.

Part V

Monoid Algebras

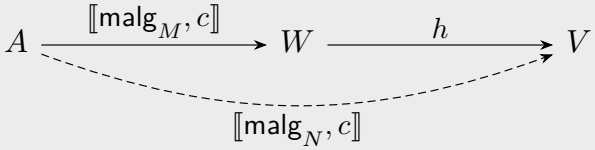
Monoid Algebras




```
Definition malg_ (leaf : Sh ~> option W) (x : App F W) : W :=
  let (s, k) := x in mconcat M (opt_cons (leaf s) (map k (members F s))).
```

One Fusion Law for Every Monoid Algebra

```
Lemma malg_fusion (Hh : MonoidHom M N h)
  : h \o hylo (malg F M leaf) c =e hylo (malg F N (omap h \o leaf)) c.
Proof. apply hylo_fusion_l, malg_hom. Qed.
```



Maximum Segment Sum

```
Definition mss := maximum \o lmap sum \o segs \o lmap some_m.

Derive mss_linear in (mss =e morph_of mss_linear) as mss_calc.
Proof.
  unfold mss, segs.
  arewrite (map_promotion sum).
  arewrite (reduce_promotion max_monoid).
  arewrite (lmap_comp maximum (lmap sum)).
  arewrite (lmap_comp (maximum \o lmap sum) inits).
  rewrite horner_mss.                          (* max-plus semiring *)
  arewrite (scan_lemma odot (Some 0)).
  arewrite (scan_tupling max_monoid odot (Some 0)).
  arewrite (foldr_lmap (tup_step max_monoid odot) some_m (Some 0, Some 0)).
  finished.
Qed.
```

Maximum Segment Sum

```
let mss_linear x =
  let (y, _) =
    let rec f = function
    | [] -> ((Some Bigint.zero), (Some Bigint.zero))
    | h :: t ->
      let p = f t in
      let z0 = omax (Some Bigint.zero) (oplus (Some h) (snd p)) in
      ((omax z0 (fst p)), z0)
    in f x
  in
  y
```

Wrap-up

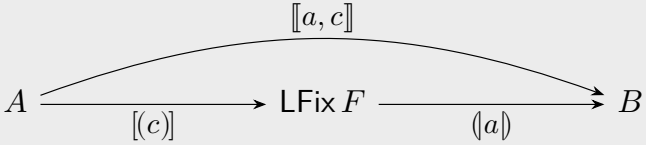
Summary

- Hylomorphisms in Rocq: axiom-free, extractable.
- Laws are theorems; calculations are `rewrite` proofs.
- Termination proved once per coalgebra.
- Idiomatic OCaml, down to native arrays.

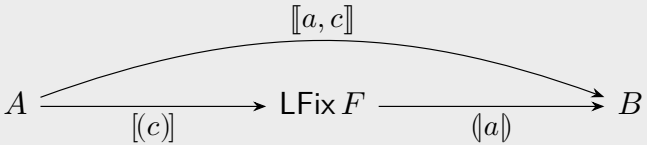
What Changed Since the Paper

- Rocq 9; properness and termination by instance search.
- `Derive` and `arewrite` for calculations.
- Finite containers and monoid algebras.
- New examples: MSS, fast power, knapsack with arrays.

Open Question 1: Which Intermediate Structure?

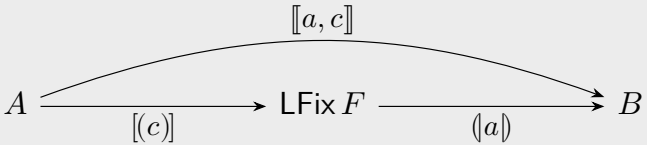


Open Question 1: Which Intermediate Structure?



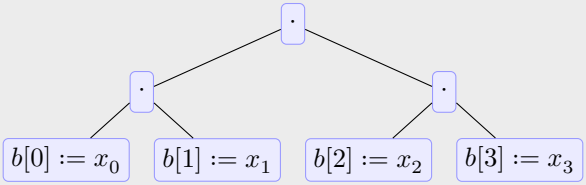
- A law that splits through a **buffer**, not a tree?

Open Question 1: Which Intermediate Structure?



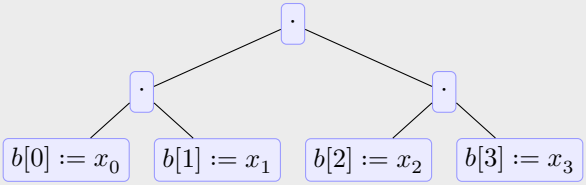
- A law that splits through a **buffer**, not a tree?
- Fused or split: a question of **cost**.

Open Question 2: Algebras that Write



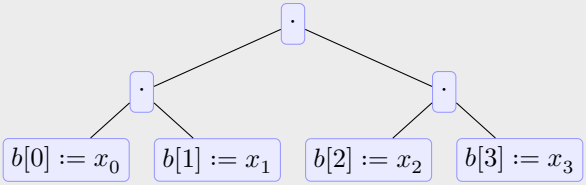
- Writes form a monoid: `malg_fusion` applies.

Open Question 2: Algebras that Write



- Writes form a monoid: `malg_fusion` applies.
- Which equivalence on buffers?

Open Question 2: Algebras that Write



- Writes form a monoid: `malg_fusion` applies.
- Which equivalence on buffers?
- Exactly once, proved per coalgebra?

Open Question 3: Programs Without Closures

- `cont` is a function: extraction gives closures.

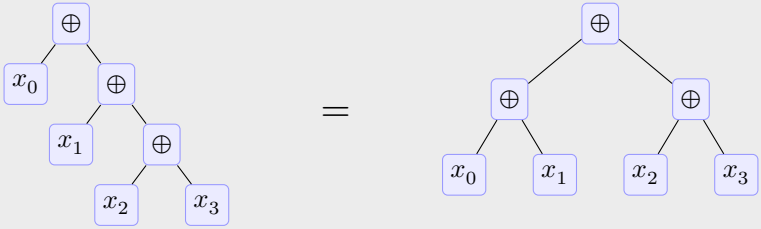
Open Question 3: Programs Without Closures

- `cont` is a function: extraction gives closures.
- A first-order syntax, **closed under the laws**?

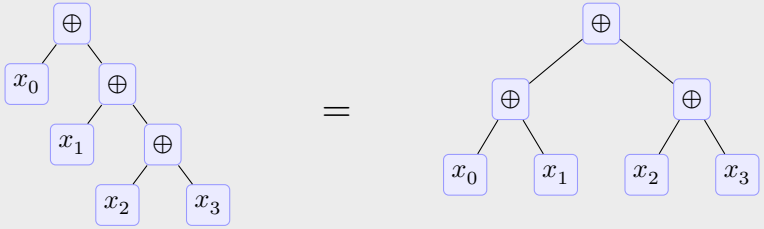
Open Question 3: Programs Without Closures

- `cont` is a function: extraction gives closures.
- A first-order syntax, **closed under the laws**?
- Where do overflow and bounds conditions live?

Open Question 4: Changing the Coalgebra

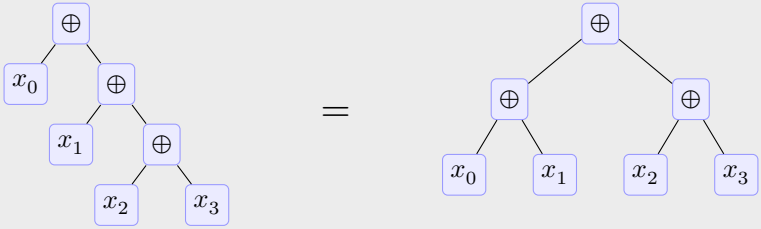


Open Question 4: Changing the Coalgebra



- A generic law: same leaves, same order, same result?

Open Question 4: Changing the Coalgebra



- A generic law: same leaves, same order, same result?
- And if the order differs?

Thank you!

Questions?

`https://github.com/dcastrop/rocq-hylomorphisms`

Based on joint work with Marco Paviotti and Michael Vollmer.